Documentación completa — Ahorra y Prospera

Este documento explica todo el proyecto en un solo lugar: qué hace cada archivo, cómo se publicó en internet, dónde está la base de datos, cómo se obtuvo la dirección (URL) en Render, cómo se usa la inteligencia artificial, dónde interviene n8n y cómo se envían los correos electrónicos a cualquier dirección.

Contenido

1. Qué es la aplicación y cómo está armada
2. Qué hace cada archivo
3. Cómo se hizo el despliegue (GitHub, Aiven, Render y la URL)
4. Cómo se usa la inteligencia artificial
5. Dónde y cómo se usa n8n
6. Cómo se envían los correos electrónicos
7. Cómo se logró enviar a cualquier correo (Gmail, Yahoo, Hotmail, institucionales)
8. Limitaciones y cosas que conviene saber

---

1. Qué es la aplicación y cómo está armada

"Ahorra y Prospera" es una aplicación web de finanzas personales: permite registrar ingresos y gastos, crear presupuestos, metas de ahorro, deudas, inversiones, retos, lecciones de educación financiera, un plan financiero y reportes en Excel y PDF.

Tiene tres piezas:

Pieza	Tecnología	Dónde está en el proyecto
Frontend (lo que ve el usuario)	HTML, CSS y JavaScript puro, sin frameworks	carpeta `frontend/`
Backend (la lógica y la API)	Node.js con Express	carpeta `backend/`
Base de datos	MySQL	en tu computador con XAMPP; en internet con Aiven

El navegador pide los datos al backend con `fetch`, enviando un token de sesión en la cabecera `Authorization: Bearer <token>`. El backend consulta MySQL y responde.

```
Navegador ──fetch──► Express (Render) ──SQL──► MySQL (Aiven)
                            │
                            ├──► Correo (Google Apps Script con Gmail)
                            ├──► IA (Gemini / Groq-Llama / Claude)
                            └──► n8n (webhooks de automatización)
```

Cómo arranca el backend: `app.js` carga el `.env`, se conecta a MySQL, crea las tablas si la base está vacía, ejecuta las migraciones, levanta el servidor y programa una revisión de notificaciones cada hora. El servidor registra solo todas las rutas que encuentre en `src/routes/<carpeta>/<archivo>.js`: cada archivo queda publicado bajo `/<nombre-del-archivo>` (por ejemplo `routes/finanzas/gastos.js` queda en `/gastos`).

---

2. Qué hace cada archivo

2.1 Raíz del proyecto

Archivo	Qué hace
`README.md`	Resumen del proyecto, cómo ejecutarlo y cómo configurar Google, correo, IA y n8n.
`render.yaml`	Receta de despliegue para Render: carpeta `backend`, comando de instalación, comando de arranque, chequeo de salud y lista de variables de entorno.
`.gitignore`	Lista lo que Git no sube a GitHub (`.env`, `node_modules`, fotos subidas).
`.vscode/settings.json`	Fija el puerto de Live Server en 5501.
`readme/guia-del-proyecto.md`	Guía técnica anterior, con más detalle de algunas secciones.
`readme/n8n.md`	Detalle de la integración con n8n.
`readme/documentacion-completa.md`	Este documento.
`docs/n8n/`	Tres flujos de ejemplo para importar en n8n (ver sección 5).

2.2 Backend — raíz de `backend/`

Archivo	Qué hace
`app.js`	Punto de entrada: conecta a MySQL, crea tablas si faltan, corre migraciones, inicia Express y arranca el revisor horario.
`.env`	Configuración secreta de tu computador (base de datos, claves de IA, correo, n8n). **No se sube a GitHub.**
`.env.example`	Plantilla del `.env` con todos los nombres de variables y sin valores. Sí se sube.

Archivo	Qué hace
`package.json`	Dependencias (express, mysql2, cors, express-fileupload, nodemailer, exceljs, pdfkit, dotenv) y comandos (`npm start` , `npm run dev` , `npm run probar-config` , `npm run exportar-bd` , `npm run importar-bd`).
`package-lock.json` , `pnpm-lock.yaml`	Versiones exactas de las dependencias. Hay dos porque se mezclaron gestores; conviene usar solo uno (npm).
`scripts/probar-config.mjs`	Diagnóstico: prueba cada IA configurada y envía un correo de prueba (`npm run probar-config correo@dominio.com`).
`scripts/enviar-correo.gs`	Código del Google Apps Script que envía los correos con tu Gmail (sección 7). **No se ejecuta en el servidor**: se pega en script.google.com.
`database/esquema.sql`	Descripción de todas las tablas, vistas, triggers y datos de referencia (monedas, categorías, logros, retos, tipos de inversión, lecciones).
`database/exportar.mjs`	Genera `esquema.sql` desde tu base local (`npm run exportar-bd`). No incluye usuarios ni datos personales.
`database/importar.mjs`	Crea todo el esquema en la base configurada en `MYSQL_*` (`npm run importar-bd`).
`uploads/`	Carpeta donde se guardan las fotos de perfil.

2.3 Backend — `src/config` , `src/server` , `src/middleware` , `src/utils`

Archivo	Qué hace
`config/env.js`	Carga el `.env` y fija la zona horaria `America/Bogota` .
`config/mysql/dbmysql.js`	Crea el *pool* de conexiones MySQL, activa SSL si `MYSQL_SSL=true` y prueba la conexión al arrancar.
`server/express.server.js`	Configura Express (CORS, JSON, subida de archivos hasta 5 MB), sirve `/uploads` y la carpeta `frontend/` como sitio web, carga las rutas y usa `trust proxy` para funcionar detrás de Render.
`middleware/sesion.js`	`requiereSesion` : lee el token, busca la sesión en la tabla `sesiones` , comprueba que el usuario esté activo y deja el usuario en `req.usuario` . Si no es válido responde 401.
`utils/contrasenas.js`	Cifra y verifica contraseñas con scrypt y sal aleatoria.
`utils/validar.js`	Validaciones compartidas (fechas, números, textos, errores 400/500).
`utils/fotos.js`	Define la carpeta de fotos y responde 404 si no existe.
`utils/movimientosRouter.js`	Fábrica de las rutas de ingresos y gastos (listar, crear, editar, borrar). Al guardar revisa logros y alertas y avisa a n8n.

2.4 Backend — `src/routes` (la API)

Archivo	Qué hace
`index.js`	Cargador automático de rutas.
`api/salud.js`	`GET /salud` : comprueba que el servidor y la base responden.
`auth/auth.js`	Registro, inicio de sesión, cierre de sesión, `GET /auth/yo` , monedas y acceso con Google. Bloquea la cuenta tras 5 contraseñas incorrectas.
`finanzas/ingresos.js` , `gastos.js`	Rutas de ingresos y gastos (usan `movimientosRouter.js`).
`finanzas/categorias.js`	Categorías del sistema y propias del usuario.
`presupuesto/presupuestos.js`	Presupuesto mensual: total, límites por categoría, historial, sugerencias y presupuesto sugerido.
`presupuesto/pagos.js`	Pagos programados; al marcarlos como pagados crea el gasto y reprograma los recurrentes.
`ahorro/metas.js`	Metas de ahorro y aportes.
`ahorro/cuentas.js`	Cuentas de ahorro (banco, billetera, efectivo).
`ahorro/inversiones.js`	Inversiones, tipos y ganancias.
`ahorro/chat.js`	Chat con el asesor financiero con IA y límite diario.
`deudas/deudas.js`	Deudas y sus pagos.
`plan/plan.js`	Genera el plan financiero y su Excel.
`cuenta/resumen.js`	Datos del panel de Gestión Financiera en una sola llamada.
`cuenta/notificaciones.js`	Campanita: listar, marcar como leídas, eliminar.
`cuenta/retos.js` , `logros.js`	Retos, logros, puntos, niveles y recompensas.
`cuenta/educacion.js`	Lecciones, progreso, lección generada con IA y quizzes.
`cuenta/recomendaciones.js`	Recomendaciones guardadas por el plan.
`cuenta/perfil.js`	Datos del usuario, foto, cambio de moneda con conversión, contraseña y sesiones.
`cuenta/exportaciones.js`	Reportes en Excel (ExcelJS) y PDF (PDFKit) e historial de exportaciones.
`integraciones/n8n.js`	Endpoints para que n8n le escriba a la app (sección 5).

2.5 Backend — `src/servicios` (la lógica de negocio)

Archivo	Qué hace
`esquema.js`	Si la base está vacía, crea todas las tablas al arrancar (clave en el despliegue).
`migraciones.js`	Cambios pequeños a la base al iniciar, seguros de repetir.
`notificaciones.js`	Cada hora revisa a los usuarios: hereda el presupuesto, avisa de deudas y metas y llama a las alertas.
`alertas.js`	Genera las alertas (campanita y correo). Cada alerta se manda una sola vez por mes o por pago.
`correo.js`	Envío de correos (secciones 6 y 7).
`presupuestoMensual.js`	Copia el presupuesto del mes anterior al mes nuevo.
`logros.js` , `retos.js`	Logros, puntos, niveles y progreso de retos.
`planFinanciero.js`	Motor de cálculo del plan (regla 50/30/20, método avalancha, fondo de emergencia, metas, portafolio). Es matemática pura, sin IA.
`planExcel.js` , `reportePdf.js`	Arman el Excel del plan y el PDF de reportes.
`ia.js`	Capa única para hablar con la IA (sección 4).
`cuotaIA.js`	Límite diario de consultas con IA por usuario.
`asesorInversiones.js`	Reglas de seguridad y datos del usuario para el chat; asesor básico de respaldo.
`asesorIA.js`	Redacta con IA el resumen ejecutivo del plan.
`educacionIA.js` , `contenidoEducativo.js`	Educación personalizada: lecciones, quizzes y banco de preguntas de respaldo.
`n8n.js`	Envía los eventos a n8n (sección 5).

2.6 Frontend

Páginas (`frontend/`)

Archivo	Qué es
`index.html`	Página de inicio pública.
`pages/auth/login.html` , `registro.html`	Inicio de sesión y registro (con botón de Google).
`pages/app/dashboard.html`	Gestión Financiera: resumen del mes.
`pages/app/presupuesto.html`	Presupuesto del mes y general.
`pages/app/ingresos-gastos.html`	Ingresos y gastos en pestañas.
`pages/app/ahorros.html`	Metas y cuentas de ahorro.
`pages/app/inversiones.html`	Inversiones, tipos, simulador, ganancias y asesor.

Archivo	Qué es
`pages/app/deudas.html`	Deudas y pagos programados.
`pages/app/retos.html`	Retos, logros y recompensas.
`pages/app/educacion.html`	Lecciones y actividades.
`pages/app/ia-financiera.html`	Chat con el asesor IA.
`pages/app/plan-financiero.html`	Plan financiero y recomendaciones.
`pages/app/reportes.html`	Reportes en Excel o PDF.
`pages/app/configuracion.html`	Perfil, foto, moneda, contraseña y sesiones.
`pages/app/ingresos.html` , `gastos.html`	Redirigen a `ingresos-gastos.html` .

Scripts (`frontend/assets/js/`)

Archivo	Qué hace
`session.js`	Cliente de la API, guarda el token y protege las páginas privadas.
`ui.js`	Formato de dinero y fechas, avisos, ventanas emergentes, pestañas.
`shell.js`	Tema claro/oscuro, menú de perfil y campanita de notificaciones.
`auth.js` , `landing.js`	Login/registro con Google y menú de la página de inicio.
`dashboard.js`	Panel de Gestión Financiera (gráficas con Chart.js).
`presupuesto.js` , `movimientos.js` , `ahorros.js` , `deudas.js` , `pagos.js`	Pantallas de presupuesto, ingresos/gastos, ahorro, deudas y pagos.
`inversiones.js` , `tipos-inversion.js`	Inversiones, simulador y catálogo de 22 tipos de inversión.
`retos.js` , `educacion.js`	Retos, logros, lecciones y quiz.
`chat.js` , `ia.js` , `plan.js`	Chat con IA, recomendaciones y plan financiero.
`reportes.js` , `configuracion.js`	Descarga de reportes y configuración del perfil.

Estilos e imágenes: `assets/css/site.css` (inicio, login, registro), `app.css` (panel, temas claro y oscuro), `plan.css` (pestañas, tablas, ventanas, chat), `assets/img/logo*.svg` (logos) y una imagen `.avif` que ninguna página usa.

---

3. Cómo se hizo el despliegue

Idea general: "desplegar" es poner la aplicación en computadores de internet que están encendidos todo el día, para que otras personas entren sin depender de tu computador. Hicieron falta tres servicios gratuitos:

Servicio	Para qué se usó
GitHub	Guardar el código. Render lo descarga de ahí.
Aiven	Alojar la base de datos MySQL.
Render	Ejecutar la aplicación (backend y páginas) y darle una dirección pública.

Se necesitan dos servicios (Render y Aiven) porque el plan gratuito de Render no incluye una base MySQL.

3.1 Qué se preparó en el código

- `render.yaml`: le dice a Render qué carpeta usar (`backend`), cómo instalar (`npm install`), cómo arrancar (`node app.js`) y qué variables pedir.
- `backend/database/esquema.sql` y los scripts `exportar.mjs` / `importar.mjs`: describen y crean las tablas.
- `backend/src/servicios/esquema.js`: al arrancar, si la base está vacía (no existe la tabla `usuarios`), **crea todas las tablas sola**. Así no hizo falta importarlas a mano.
- `express.server.js` con `trust proxy`, para que funcione detrás del proxy de Render.
- `.gitignore`: evita subir el `.env` con las claves.
- `.env.example`: plantilla de variables.

3.2 Paso 1 — Subir el código a GitHub

El código está en https://github.com/lauratrujillo774/Ahorra-y-Prospera, en la rama `main`. Para subir cambios:

```
git add .
git commit -m "describe el cambio"
git push
```

El archivo `.env` (con las claves) **no** se sube. Esas claves se escriben después en el panel de Render.

3.3 Paso 2 — Dónde se subió la base de datos (Aiven)

1. Se creó una cuenta en https://aiven.io y un servicio **MySQL** con el plan gratuito.
2. Cuando el estado pasó a *Running*, se copiaron los cinco datos de conexión: **Host**, **Port**, **User** (normalmente `avnadmin`), **Password** y **Database name** (normalmente `defaultdb`).
3. Aiven exige conexión segura, por eso la app usa `MYSQL_SSL=true`.
4. La base de Aiven nace **vacía**. Al arrancar por primera vez en Render, la app detectó que estaba vacía y creó sola las 30 tablas, 7 vistas, 4 triggers y los catálogos (lo anunció en los Logs).

La base de Aiven empieza sin usuarios ni gastos: los datos de prueba de tu XAMPP no se copiaron.

3.4 Paso 3 — Publicar la aplicación en Render

1. Se creó la cuenta en https://render.com iniciando sesión con GitHub.
2. **New +** → **Blueprint**, se eligió el repositorio `Ahorra-y-Prospera`. Render lee `render.yaml` y ya sabe qué hacer.
3. Render pidió el valor de las variables que `render.yaml` deja vacías (`sync: false`). Se escribieron en el panel *Environment*:

Variable	Valor
`MYSQL_HOST`, `MYSQL_PORT`, `MYSQL_USER`, `MYSQL_PASSWORD`	Los de Aiven
`MYSQL_DATABASE`	El nombre de la base de Aiven (`defaultdb`), **no** `ahorra_y_prospera`
`MYSQL_SSL`	`true`
`APP_URL`	La dirección pública que dio Render
`GOOGLE_CLIENT_ID`	El ID de cliente de Google para el botón de acceso
`GEMINI_API_KEY`	La clave de la IA
`APPS_SCRIPT_URL`, `APPS_SCRIPT_CLAVE`	El envío de correos (sección 7)
`PORT`	No se pone: Render la asigna sola

4. Render **construyó** la app (descarga el código, `npm install`) y la **arrancó** con `node app.js`. El avance se ve en la pestaña *Logs*.

3.5 Cómo se sacó la URL

La dirección no se "saca" con un comando: **Render la genera sola**. Al crear el servicio con el nombre `ahorra-y-prospera` (definido en `render.yaml`), Render le asigna el subdominio gratuito de ese nombre:

```
https://ahorra-y-prospera.onrender.com
```

Cuando el estado del servicio dice **Live**, la dirección aparece arriba en el panel de Render. Esa es la dirección pública de la app. Después:

- Se copió esa dirección a la variable `APP_URL` (la usan los botones de los correos).
- Se autorizó en Google Cloud (*Credenciales → ID de cliente de OAuth → Orígenes autorizados de JavaScript*) para que funcione el botón "Acceder con Google".
- Se comprobó que `https://ahorra-y-prospera.onrender.com/salud` respondiera `{"servidor":"ok","baseDeDatos":"ok"}`. Eso confirma que Render sí llega a la base de Aiven.

Render entrega HTTPS automáticamente y cada `git push` a `main` vuelve a desplegar la app solo.

3.6 Problemas reales que aparecieron

1. `Unknown database 'ahorra_y_prospera'`: `MYSQL_DATABASE` tenía el nombre de la base local. Se corrigió con `defaultdb`.
2. `Failed to open the referenced table 'usuarios'`: la base de Aiven estaba vacía. Se añadió la creación automática de tablas (`esquema.js`).
3. Google pidió "acceso adicional a proyecto": la cuenta de Google no tenía acceso al proyecto del ID de cliente. Se resolvió cambiando de cuenta o creando un proyecto nuevo en Google Cloud.

---

4. Cómo se usa la inteligencia artificial

Aclaración importante: el proyecto **no usa OpenAI (ChatGPT)**. Usa tres proveedores: **Gemini** (Google), **Groq** (que ejecuta modelos Llama) y **Claude** (Anthropic). Groq se menciona a veces junto a OpenAI porque su API tiene el mismo formato (`/openai/v1/chat/completions`), pero el modelo es Llama (`llama-3.3-70b-versatile`). Si en la presentación necesitas hablar de OpenAI, conviene decir que **no se utilizó**.

4.1 La capa común (`servicios/ia.js`)

- Usa el primer proveedor que tenga clave en el `.env`: `GEMINI_API_KEY`, `GROQ_API_KEY` o `ANTHROPIC_API_KEY`. `IA_PROVIDER` permite elegir cuál va primero.
- **Respaldo en cadena:** si uno falla (cuota agotada, error), prueba el siguiente.
- **Gemini:** prueba una lista de modelos y usa el primero disponible, porque Google retira modelos con frecuencia.
- **Reintento:** ante errores pasajeros (500, 502, 503) reintenta una vez.
- Las llamadas son `fetch` directos a la API de cada proveedor, sin librerías adicionales.
- Las claves **nunca** van en el código: solo en el `.env` o en el panel de Render.

4.2 Límite diario (`servicios/cuotaIA.js`)

Cada consulta se anota en la tabla `consultas_ia`. Cada usuario tiene **20 consultas al día** (cambiable con `IA_LIMITE_DIARIO`) más las que gana con recompensas (Bronce +10, Plata +15, Oro +25). Al agotarse, el chat sigue funcionando con un asesor básico, sin costo.

4.3 Dónde se usa

Función	Archivos	Cómo funciona
Chat del asesor	`routes/ahorro/chat.js`, `servicios/asesorInversiones.js`, `frontend/assets/js/chat.js`	El backend arma un mensaje de sistema con reglas de seguridad (no prometer rentabilidades, no recomendar activos concretos, advertir de estafas, no inventar cifras) y le agrega los datos del usuario. Si no hay IA, responde con textos preparados.
Presupuesto sugerido	`routes/presupuesto/presupuestos.js`	**Lo calcula la propia app con reglas**, sin llamar a un modelo, aunque en pantalla se llame "con IA".

Función	Archivos	Cómo funciona
Plan financiero	`servicios/planFinanciero.js`, `servicios/asesorIA.js`	Los números los calcula el motor de reglas; la IA solo redacta el resumen ejecutivo. Sin IA, el plan se muestra igual.
Educación personalizada	`servicios/educacionIA.js`, `routes/cuenta/educacion.js`	La IA escribe una mini-lección con los números del usuario y genera quizzes de 3 preguntas en JSON, que se validan. Si falla, se usa un banco de preguntas preparadas.

La IA es una ayuda educativa, no un asesor financiero certificado; las pantallas lo avisan.

---

5. Dónde y cómo se usa n8n

n8n es una herramienta de automatización. La app se conecta con ella en **dos direcciones**, y todo es opcional: si las variables están vacías, la integración queda apagada y la app funciona igual.

Variable (`backend/.env`)	Para qué sirve
`N8N_WEBHOOK_URL`	URL del nodo *Webhook* del flujo en n8n. Vacía = no se envía nada.
`N8N_SECRET`	Frase con la que se firma cada aviso (HMAC-SHA256) para que n8n compruebe que viene de la app.
`N8N_API_KEY`	Clave (mínimo 16 caracteres) que n8n debe enviar en la cabecera `x-api-key` para usar la API de entrada.

5.1 Salida: la app avisa a n8n

`servicios/n8n.js` tiene la función `emitirEvento(evento, datos)`: envía un POST JSON firmado, en segundo plano, con tiempo límite de 5 segundos y sin romper la app si n8n falla. Se dispara en estos lugares:

Evento	Cuándo ocurre	Archivo
`usuario_registrado`	Alguien crea su cuenta (con correo o con Google)	`routes/auth/auth.js`
`alerta`	Se genera una alerta de presupuesto, ingresos o pagos	`servicios/alertas.js`
`presupuesto_generado`	Se guarda un presupuesto hecho con IA	`servicios/alertas.js`
`logro`	El usuario gana una insignia	`servicios/logros.js`
`pago_realizado`	Se marca un pago programado como pagado	`routes/presupuesto/pagos.js`

Evento	Cuándo ocurre	Archivo
`ingreso_registrado`, `gasto_registrado`	Se registra un ingreso o gasto	`utils/movimientosRouter.js`
`meta_cumplida`	Un aporte completa una meta	`routes/ahorro/metas.js`
`deuda_pagada`	Una deuda queda saldada	`routes/deudas/deudas.js`
`reto_completado`	Un reto llega al 100 %	`servicios/retos.js`
`recompensa_reclamada`	Se reclama una recompensa	`routes/cuenta/logros.js`
`ganancia_inversion`	Se registra una ganancia	`routes/ahorro/inversiones.js`
`plan_generado`	Se genera un plan financiero	`routes/plan/plan.js`
`reporte_generado`	Se descarga un reporte en Excel o PDF	`routes/cuenta/exportaciones.js`

5.2 Entrada: n8n le habla a la app

Los endpoints están en `routes/integraciones/n8n.js`, publicados bajo `/n8n` y protegidos con la cabecera `x-api-key`:

Endpoint	Para qué sirve
`GET /n8n/estado`	Comprobar la conexión
`POST /n8n/notificar`	Crear una notificación en la campanita de un usuario
`GET /n8n/resumen-semanal`	Ingresos, gastos y balance de cada usuario (para correos semanales)
`POST /n8n/ingreso`, `POST /n8n/gasto`	Registrar movimientos desde otra app (Telegram, WhatsApp, Sheets)
`POST /n8n/aporte-meta`	Aportar a una meta de ahorro
`GET /n8n/usuario`	Foto rápida del mes de un usuario para un bot
`GET /n8n/pagos-proximos`	Pagos que vencen pronto, para recordatorios
`GET /n8n/usuarios-en-riesgo`	Usuarios que ya gastaron el 80 % o más de sus ingresos

5.3 Flujos de ejemplo (`docs/n8n/`)

Archivo	Qué hace
`flujo-ahorra-y-prospera.json`	Recibe los eventos, envía un correo de bienvenida con Gmail a los usuarios nuevos y deja un eco en la campanita.
`flujo-telegram-registrar-gastos.json`	Escribes `gasto 25000 almuerzo` en Telegram y se registra en la app; `resumen` responde cómo vas en el mes.
`flujo-resumen-semanal-y-recordatorios.json`	Cada lunes a las 8:00 envía el resumen semanal por correo; cada día a las 9:00 recuerda los pagos que vencen en 2 días.

5.4 Qué se probó y qué no (dicho con honestidad)

- **Probado:** el envío firmado de cada evento y los endpoints de entrada (clave correcta e incorrecta, validaciones, registro de movimientos), usando un servidor n8n **simulado**.
- **No probado:** con una instalación real de n8n ni con credenciales reales de Gmail o Telegram. Los tres flujos son archivos JSON válidos, pero hay que importarlos y ajustarlos en tu n8n (cambiar `PON_AQUI_TU_N8N_API_KEY` y conectar tus credenciales).
- Que n8n esté o no conectado en la versión publicada en Render depende de si se pusieron `N8N_WEBHOOK_URL` y `N8N_API_KEY` en su panel. Si están vacías, la integración está apagada.

---

6. Cómo se envían los correos electrónicos

Todo está en `backend/src/servicios/correo.js`, y quien decide cuándo enviar es `servicios/alertas.js`.

6.1 Piezas

1. `correoConfigurado()`: comprueba si hay algún método de envío configurado. Si no lo hay, la app **no falla**: simplemente no envía correos y las alertas llegan a la campanita.
2. `plantillaAlerta()`: arma el correo en HTML (encabezado verde con el nombre de la app, título, mensaje, tabla opcional y botón "Ver mi presupuesto" que apunta a `APP_URL`). Todo el texto se **escapa** para evitar HTML malicioso.
3. `enviarCorreo({ para, asunto, html, texto })`: envía el mensaje y **nunca lanza error**; si falla, lo registra en consola para que un problema de correo no rompa la app.

6.2 Cuándo se envían

- "Te queda poco de tus ingresos" (80 % gastado) y "gastaste todos tus ingresos".
- Presupuesto total al llegar al porcentaje de aviso y al superarlo.
- Límite de una categoría al llegar a su aviso y al superarlo.
- Pago programado que vence en 2 días o menos.
- Presupuesto generado con IA (con tabla por categoría).

6.3 Cómo se evita el spam

`alertar()` consulta primero si ya existe una notificación con el mismo título ese mes; si existe, no repite ni la notificación ni el correo. Cada alerta crea la notificación en la campanita y manda el correo en segundo plano, sin hacer esperar a la app.

6.4 Cuándo se evalúan

Al registrar o editar un ingreso o gasto, al guardar un presupuesto, al programar un pago, al consultar las notificaciones y **cada hora** automáticamente (`servicios/notificaciones.js`).

---

7. Cómo se logró enviar a cualquier correo

7.1 El destinatario nunca estuvo limitado

Cada alerta se envía a la dirección que el usuario escribió al registrarse (`usuario.correo`). El código **no filtra por dominio**: un Gmail, un Yahoo, un Hotmail, un `@iesantaelena.edu.co` o un `@elpoli.edu.co` se tratan exactamente igual. La validación al registrarse solo comprueba que el texto tenga forma de correo (`algo@dominio.ext`).

7.2 El verdadero problema era la forma de enviar

Al principio los correos salían por **SMTP de Gmail** con la librería Nodemailer. Eso tiene dos problemas:

- **Render bloquea la salida SMTP** en su plan gratuito, así que en internet los correos no salían.
- Gmail por SMTP exige una "contraseña de aplicación" y tiene topes de envío.

7.3 La solución: un Google Apps Script

Brevo (servicio de correo por API) se probó como alternativa, pero **no es gratuito para este uso**, así que se descartó como opción principal. La solución elegida es gratuita y funciona en Render:

1. Un pequeño programa de Google (**Google Apps Script**) vive en tu cuenta de Google. Su código está en `backend/scripts/enviar-correo.gs` .
2. Ese script se publica como "Aplicación web": recibe una petición por internet (HTTPS) y, con la función `GmailApp.sendEmail` , envía el correo desde tu Gmail a **cualquier dirección**.
3. El backend (`correo.js` , función `enviarPorAppsScript`) hace un `fetch` POST a la URL del script con un JSON: `{ clave, para, asunto, html, texto }` .
4. Como es una petición HTTPS normal (no SMTP), **Render no la bloquea**.
5. El script comprueba la `clave` secreta (para que nadie más pueda usar tu URL para enviar correos) y que el destinatario tenga forma de correo.

Orden en que `correo.js` elige cómo enviar: Google Apps Script (si existen `APPS_SCRIPT_URL` y `APPS_SCRIPT_CLAVE`) → Brevo (si existen `BREVO_API_KEY` y `BREVO_FROM`) → SMTP (si existen `SMTP_HOST` , `SMTP_USER` y `SMTP_PASS`).

7.4 Cómo se configura (una sola vez)

1. Entra a script.google.com y crea un proyecto nuevo.
2. Pega el contenido de `backend/scripts/enviar-correo.gs` y cambia `CAMBIA-ESTA-CLAVE` por una frase larga y secreta.
3. *Implementar → Nueva implementación → Aplicación web*. Ejecutar como: **Yo**. Acceso: **Cualquier persona**. Acepta los permisos de Gmail.
4. Copia la URL que termina en `/exec` .
5. Ponla en `backend/.env` y en el panel *Environment* de Render:

```
APPS_SCRIPT_URL = https://script.google.com/macros/s/XXXX/exec
APPS_SCRIPT_CLAVE = <la misma frase del script>
```

6. Prueba: `npm run probar-config alguien@iesantaelena.edu.co`.

7.5 Estado actual

El código para enviar con Apps Script **ya está escrito**, pero **no se ha probado con un script real** porque solo tú puedes crearlo en tu cuenta de Google. Hasta que pongas `APPS_SCRIPT_URL` y `APPS_SCRIPT_CLAVE`, los correos no salen desde Render y las alertas llegan solo a la campanita. Prueba primero con tu propio correo y luego con uno de cada tipo (Gmail, Hotmail, institucional).

---

8. Limitaciones y cosas que conviene saber

- **La app se duerme:** Render gratuito la suspende tras unos 15 minutos sin visitas; la primera visita después tarda cerca de un minuto. Mientras duerme **no corre la revisión horaria** de alertas.
- **Fotos de perfil:** el disco gratuito de Render es temporal; las fotos subidas se pierden al reiniciar o redesplegar.
- **Tope de correos:** una cuenta de Gmail gratuita permite unos 100 correos al día desde Apps Script.
- **Spam:** los correos salen desde tu Gmail; algunos dominios institucionales pueden mandarlos a spam. Pide que marquen la dirección como segura.
- **Aiven y Render gratuitos:** sus límites y precios cambian; confírmalos en sus páginas.
- **Seguridad:** nunca subas el `.env`; usa claves nuevas si alguna apareció en un chat; usa contraseñas largas en Aiven y en `N8N_API_KEY`.
- **Para solo mostrarla:** si no necesitas que otras personas entren con tu computador apagado, puedes ejecutarla en local (`npm run dev` + XAMPP) y abrir `http://localhost:5000`.